

PX4 Fixed-Wing Autosoaring Implementation

Complete Technical Defence Report

Gliding and Thermalling Modes: Architecture, Mathematics, and Implementation

Six-Part Report Series

Radhouene — Fixed-Wing Autosoaring Project

April 2026

Abstract

This document is a complete technical report of the autosoaring implementation for a PX4-based fixed-wing UAV, covering engine-off gliding and thermal soaring. The work modifies six subsystems of the PX4 autopilot: the uORB/DDS inter-process communication layer, the Total Energy Control System (TECS), the Fixed-Wing Position Control module, the Commander, the Navigator, and the ROMFS airframe and parameter definitions.

Each of the six reports below presents: (i) the original system design and mathematical model, (ii) the motivation for modification, and (iii) the complete modified design with equations, physics, and code.

This document is intended to prepare the author for a thesis defence by providing a self-contained, mathematically rigorous reference.

Contents

I	Inter-Process Communication and Message Design	5
1	uORB Messaging and XRCE-DDS Bridge	6
1.1	Introduction	6
1.2	Original System: uORB and the Absence of Soaring IPC	6
1.2.1	uORB Publish-Subscribe Middleware	6
1.2.2	XRCE-DDS Bridge	7
1.3	Modified System: Autos soaringControl Message Design	7
1.3.1	Design Rationale	7
1.3.2	Complete Autos soaringControl Message	7
1.3.3	Field Usage Matrix	8
1.3.4	Staleness Watchdog	8
1.3.5	Summary	8
II	Total Energy Control System	9
2	TECS: Original Architecture and Soaring Extensions	10
2.1	Introduction	10
2.2	Original TECS: Mathematical Foundation	10
2.2.1	Specific Energy Definitions	10
2.2.2	Throttle Control Law	10
2.2.3	Pitch Control Law	11
2.2.4	Airspeed Setpoint Computation	11
2.2.5	Energy Weighting	11
2.2.6	Underspeed Detection	11
2.3	Modified TECS: Soaring-Mode Extensions	12
2.3.1	Throttle Integrator Decay During Glide	12
2.3.2	Energy Weighting Override in Glide	12
2.3.3	Glide Polar and Best-Glide Speed (Mode 2)	12
2.3.4	Banked Stall Speed Floor (Thermalling)	12
2.3.5	Underspeed Detection Bypass	13
2.3.6	Load-Factor STE Correction Disabled in Glide	13
2.3.7	Pitch Integrator Partial Reset on Entry and Exit	13
2.3.8	Summary of TECS Modifications	14

III	Fixed-Wing Position Control	15
3	FixedwingPositionControl: Soaring Dispatch and Guidance	16
3.1	Original Architecture	16
3.1.1	Module Role	16
3.1.2	NPFG Guidance	16
3.1.3	adapt_airspeed_setpoint	16
3.2	Soaring Mode Dispatch Logic	16
3.2.1	Mode Derivation	16
3.2.2	Altitude Ceiling and Floor	17
3.3	Throttle Ramps	17
3.3.1	Entry Ramp (Powered → Glide)	17
3.3.2	Exit Ramp (Glide → Powered)	17
3.4	Airspeed Setpoints per Mode	17
3.5	Mode 3: Thermal Loiter via Navigator	18
3.5.1	Loiter Radius	18
3.5.2	Signed Radius Encoding	18
3.5.3	Pre-Roll Safety (T3)	18
3.6	Mode 4: Direct Bank-Angle Thermal Guidance	18
3.6.1	Why Bypass NPFG	18
3.6.2	Roll Sign Convention	18
3.6.3	Attitude Setpoint Composition	18
3.6.4	Summary	19
IV	Commander: CLI Mode Switching	20
4	Commander Soaring Commands	21
4.1	Original System	21
4.2	Modified System: CLI Commands	21
4.2.1	Commands Added	21
4.2.2	Command Structure	21
4.2.3	Local-Override Flag	21
V	Navigator: Loiter Direction Control	22
5	Navigator DO_REPOSITION Extension	23
5.1	Original Handler	23
5.2	Modified Handler	23
5.2.1	Circular Flight Mathematics	23
5.3	Direction-Change Trigger	24
VI	ROMFS and Parameter Definitions	25
6	Soaring Parameters and Airframe Configuration	26
6.1	Parameter System Overview	26
6.2	Removed Parameter: FW_GLIDE_MODE	26

6.3	New Parameters	26
6.3.1	FW_GLIDE_AIRSPD (10.0 m/s EAS)	26
6.3.2	FW_POLAR_A (0.003 s/m) and FW_POLAR_B (0.5 m/s)	26
6.3.3	FW_GLIDE_I_DECAY (10.0 s)	27
6.3.4	FW_GLIDE_RAMP_T (3.0 s)	27
6.3.5	FW_THERMAL_BANK (40 deg)	27
6.3.6	FW_ALT_MIN (50 m), FW_ALT_MAX (200 m), FW_ALT_HYST (20 m)	27
6.4	ROMFS Airframe Defaults	27
6.5	Summary	27
A	ROS 2 Command Reference	28

List of Tables

Part I

Inter-Process Communication and Message Design

Chapter 1

uORB Messaging and XRCE-DDS Bridge

1.1 Introduction

This report describes the inter-process communication (IPC) infrastructure used to command the autosoaring system from a ROS 2 companion computer to the PX4 flight management unit (FMU). It covers:

- the uORB publish-subscribe middleware;
- the XRCE-DDS bridge connecting ROS 2 to uORB;
- the original messaging approach (boolean flags and a hidden parameter);
- the redesigned `AutosoaringControl` message using a single enumerated mode integer.

1.2 Original System: uORB and the Absence of Soaring IPC

1.2.1 uORB Publish-Subscribe Middleware

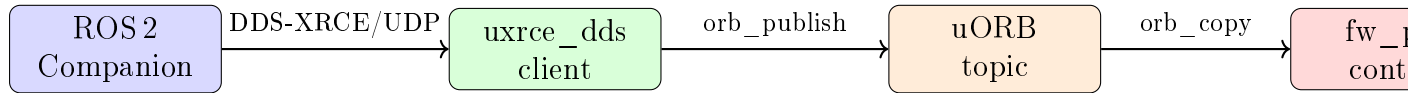
PX4 uses *uORB* (micro Object Request Broker) as its internal IPC bus. It is a strongly-typed, zero-copy, publish-subscribe system running entirely inside the NuttX RTOS. Each *topic* is defined by a `.msg` file and compiled into a C struct with an 8-byte timestamp header.

Key properties:

- **Single writer, multiple readers.**
- **Non-blocking API:** `orb_check()` tests for new data; `px4_poll()` blocks.
- **Fixed-size structs:** no dynamic allocation.

1.2.2 XRCE-DDS Bridge

The `uxrce_dds_client` module implements the DDS-XRCE protocol. Topics listed under subscriptions in `dds_topics.yaml` flow from ROS 2 into the FMU via `/fmu/in/`; topics under publications flow out.



In the original *Music* codebase, no soaring topic was registered.

1.3 Modified System: AutosoaringControl Message Design

1.3.1 Design Rationale

The first prototype used two boolean fields: `glide_mode_enabled` and `thermal_mode_enabled`. This creates four possible combinations, one of which (both true) is ambiguous. A separate PX4 parameter `FW_GLIDE_MODE` selected between fixed and polar glide, creating a hidden third dimension invisible in the message.

The redesign encodes all five states in a single `uint8 soaring_mode`:

Value	Constant	TECS	Navigator
0	SOARING_OFF	powered	unchanged
1	SOARING_GLIDE_FIXED	gliding, fixed EAS	AUTO_MISSION
2	SOARING_GLIDE_POLAR	gliding, polar EAS	AUTO_MISSION
3	SOARING_THERMAL_LOITER	gliding	AUTO_LOITER (navigator)
4	SOARING_THERMAL_BANK	gliding	AUTO_LOITER (roll override)

1.3.2 Complete AutosoaringControl Message

```

1 uint64 timestamp # microseconds since boot
2
3 uint8 soaring_mode # active mode
4 uint8 SOARING_OFF = 0
5 uint8 SOARING_GLIDE_FIXED = 1
6 uint8 SOARING_GLIDE_POLAR = 2
7 uint8 SOARING_THERMAL_LOITER = 3
8 uint8 SOARING_THERMAL_BANK = 4
9
10 # Loiter geometry (mode 3 only)
11 float32 loiter_radius_m # [m]; NaN => bank-derived minimum
12 float32 loiter_lat # [deg]; NaN/0 => current position
13 float32 loiter_lon # [deg]; NaN/0 => current position
14
15 # Optimisation (NaN => PX4 default)
16 float32 airspeed_cmd # [m/s EAS] mode 1 only
17 float32 bank_angle_cmd # [deg] modes 3,4
18 bool loiter_clockwise # true=CW, false=CCW
  
```

```

19
20 # Metadata
21 uint8    soaring_phase          # companion-reported phase
22 uint8 SOARING_PHASE_CRUISE      = 0
23 uint8 SOARING_PHASE_GLIDING     = 1
24 uint8 SOARING_PHASE_SEARCHING   = 2
25 uint8 SOARING_PHASE_CENTERING   = 3
26 uint8 SOARING_PHASE_THERMAL     = 4

```

Listing 1.1: msg/AutoSoaringControl.msg (new file)

1.3.3 Field Usage Matrix

Field	Mode 0	Mode 1	Mode 2	Mode 3	Mode 4
airspeed_cmd	—	✓	—	—	—
loiter_radius_m	—	—	—	✓	—
loiter_lat/lon	—	—	—	✓	—
bank_angle_cmd	—	—	—	✓	✓
loiter_clockwise	—	—	—	✓	✓

1.3.4 Staleness Watchdog

If the companion is silent for more than 2s while a soaring mode is active:

$$\Delta t_{\text{silence}} > 2\text{s} \Rightarrow \text{soaring_mode} \leftarrow 0 \quad (1.1)$$

This prevents the aircraft from remaining in engine-off glide indefinitely if the companion crashes or the DDS link fails.

1.3.5 Summary

Aspect	Original	Modified
Mode encoding	Two booleans + hidden param	Single <code>uint8</code> <code>soaring_mode</code>
DDS registration	Not present	<code>/fmu/in/autosoaring_control</code>
Turn direction	Not supported	<code>loiter_clockwise</code> bool
Staleness guard	Not present	2s watchdog

Part II

Total Energy Control System

Chapter 2

TECS: Original Architecture and Soaring Extensions

2.1 Introduction

TECS (Total Energy Control System) is the primary flight envelope controller for fixed-wing aircraft in PX4, originally proposed by Lambregts (1983). It controls airspeed and altitude simultaneously through two decoupled channels: throttle controls the total specific energy, and pitch controls the energy balance.

2.2 Original TECS: Mathematical Foundation

2.2.1 Specific Energy Definitions

Let m be the aircraft mass, g gravitational acceleration, V the true airspeed, and h the altitude. The specific (per unit weight) energies are:

$$E_k = \frac{V^2}{2g} \quad [\text{m}] \quad (\text{specific kinetic energy}) \quad (2.1)$$

$$E_p = h \quad [\text{m}] \quad (\text{specific potential energy}) \quad (2.2)$$

$$E = E_k + E_p = \frac{V^2}{2g} + h \quad [\text{m}] \quad (2.3)$$

Their time derivatives (specific energy rates):

$$\dot{E}_k = \frac{V\dot{V}}{g} \quad (2.4)$$

$$\dot{E}_p = \dot{h} \quad (2.5)$$

$$\dot{E} = \frac{V\dot{V}}{g} + \dot{h} \quad (2.6)$$

2.2.2 Throttle Control Law

The STE rate setpoint is:

$$\dot{E}^* = \dot{E}_k^* + \dot{E}_p^* + K_{\text{lf}}(n_z - 1) \quad (2.7)$$

where $n_z = 1/\cos\phi$ is the load factor and K_{lf} is the load-factor correction gain. The throttle demand:

$$\delta_t = \frac{\dot{E}^* - \hat{E}}{c_1} K_{\text{damp}} + \delta_{t,\text{pred}} + I_t \quad (2.8)$$

The integrator:

$$\dot{I}_t = K_{i,t}(\dot{E}^* - \hat{E}) \quad (2.9)$$

2.2.3 Pitch Control Law

Energy balance rate setpoint:

$$\dot{B}^* = w_{\text{ske}} \dot{E}_k^* - w_{\text{spe}} \dot{E}_p^* \quad (2.10)$$

Pitch angle setpoint:

$$\theta^* = \frac{1}{V} \left[K_\theta(\dot{B}^* - \hat{B}) + K_d \ddot{B} + I_\theta + \theta_{\text{ff}} \right] \quad (2.11)$$

Pitch integrator:

$$\dot{I}_\theta = K_{i,\theta}(\dot{B}^* - \hat{B}) \quad (2.12)$$

2.2.4 Airspeed Setpoint Computation

EAS to TAS conversion:

$$k_{\text{EAS2TAS}} = \sqrt{\frac{\rho_0}{\rho}}, \quad \rho_0 = 1.225 \text{ kg/m}^3 \quad (2.13)$$

Powered-flight TAS setpoint:

$$V_{\text{TAS}}^* = \text{lerp}(k_{\text{EAS2TAS}} V_{\text{EAS}}^*, V_{\text{TAS,max}}, \alpha_{\text{fd}}) \quad (2.14)$$

2.2.5 Energy Weighting

The speed weight $w_{\text{ske}} \in [0, 2]$ (parameter `FW_T_SPDWEIGHT`) adjusts channel priority. $w_{\text{spe}} = 2 - w_{\text{ske}}$.

2.2.6 Underspeed Detection

$$r_u = \frac{V_{\text{TAS,min}} - V_{\text{TAS}}}{V_{\text{TAS,min}} \varepsilon} \quad (2.15)$$

When $r_u > 0$:

$$\delta_t \leftarrow r_u \delta_{t,\text{max}} + (1 - r_u) \delta_t \quad (2.16)$$

2.3 Modified TECS: Soaring-Mode Extensions

2.3.1 Throttle Integrator Decay During Glide

To prevent throttle surge on glide exit, the integrator decays:

$$\boxed{I_t(t) = I_t(0) e^{-t/\tau}} \quad (2.17)$$

Discrete-time update ($\alpha_\tau = e^{-\Delta t/\tau}$):

$$I_t[k+1] = \alpha_\tau I_t[k] \quad (2.18)$$

For $\tau = 10$ s, $\Delta t = 20$ ms: $\alpha_\tau = e^{-0.002} \approx 0.9980$.

2.3.2 Energy Weighting Override in Glide

Since throttle is zero, all energy management is through pitch:

$$\boxed{w_{\text{spe}} = 0, \quad w_{\text{ske}} = 2} \quad (2.19)$$

The energy balance becomes:

$$\dot{B} = 2 \frac{V\dot{V}}{g} \quad (2.20)$$

2.3.3 Glide Polar and Best-Glide Speed (Mode 2)

Parabolic glide polar:

$$w_s(V) = aV^2 + b \quad (2.21)$$

Best-glide EAS (maximises range = V/w_s):

$$\boxed{V_{\text{EAS,polar}}^* = \sqrt{\frac{b}{a}}} \quad (2.22)$$

Maximum glide ratio:

$$\left(\frac{L}{D}\right)_{\text{max}} = \frac{1}{2\sqrt{ab}} \quad (2.23)$$

2.3.4 Banked Stall Speed Floor (Thermalling)

Load factor at bank angle ϕ :

$$n_z = \frac{1}{\cos \phi} \quad (2.24)$$

Banked stall speed:

$$V_{\text{stall,banked}} = V_{\text{stall,level}} \sqrt{n_z} \quad (2.25)$$

TECS stall floor:

$$\boxed{V_{\text{TAS,banked,min}} = V_{\text{TAS,min}} \sqrt{n_z}} \quad (2.26)$$

Airspeed setpoint with floor:

$$V_{\text{TAS}}^* = \max(\text{clamp}(k_{\text{EAS2TAS}} V_{\text{EAS}}^*, V_{\text{TAS,min}}, V_{\text{TAS,max}}), V_{\text{TAS,banked,min}}) \quad (2.27)$$

Example ($\phi = 40^\circ$, $V_{\text{TAS,min}} = 9 \text{ m/s}$):

$$n_z = 1 / \cos 40^\circ = 1.305 \quad (2.28)$$

$$V_{\text{TAS,banked,min}} = 9 \sqrt{1.305} \approx 10.3 \text{ m/s} \quad (2.29)$$

2.3.5 Underspeed Detection Bypass

In glide, the throttle is forced to zero regardless; the underspeed boost is meaningless and pollutes telemetry. Fix:

```
1 if (flag.glider_mode_enabled) {
2     _ratio_undersped = 0.0f;
3     return;
4 }
```

2.3.6 Load-Factor STE Correction Disabled in Glide

In powered turns, the STE correction compensates for induced drag:

$$\dot{E}^* += K_{\text{lf}} (n_z - 1) \quad (2.30)$$

In glide, throttle is already clamped to zero. The correction is skipped:

```
1 if (!flag.glider_mode_enabled) {
2     ste_rate.setpoint += param.load_factor_correction * (param.
3     load_factor - 1.f);
4 }
```

2.3.7 Pitch Integrator Partial Reset on Entry and Exit

Physical Motivation

On glide **entry**: I_θ holds a powered cruise bias. When $w_{\text{spe}} \leftarrow 0$, this bias commands nose-up, fighting the speed-hold objective.

On glide **exit**: I_θ holds a speed-hold bias. Power restoration rapidly adds STE; TECS corrects at maximum pitch rate:

$$\dot{\theta}_{\text{max}} = \frac{a_{z,\text{lim}}}{V} \approx \frac{7 \text{ m/s}^2}{10 \text{ m/s}} = 0.7 \text{ rad/s} \approx 40\text{--}50 \text{ deg/s} \quad (2.31)$$

Fix

Halve the integrator at both transitions:

$$I_\theta \leftarrow \frac{1}{2} I_\theta \quad (2.32)$$

```

1 if (flag.gliding_mode_enabled && !_prev_gliding_mode) {
2     _pitch_integ_state *= 0.5f;    // glide entry
3 } else if (!flag.gliding_mode_enabled && _prev_gliding_mode) {
4     _pitch_integ_state *= 0.5f;    // glide exit
5 }
6 _prev_gliding_mode = flag.gliding_mode_enabled;

```

2.3.8 Summary of TECS Modifications

Behaviour	Original	Modified
Throttle (unpowered)	Integrator uncontrolled	Hard-zero + I_t decay
Pitch weighting	From parameter	$w_{spe} = 0, w_{ske} = 2$
Airspeed setpoint	Mission EAS	Fixed (mode 1) or polar (mode 2)
Stall floor	$V_{TAS,min}$	$V_{TAS,min}\sqrt{n_z}$
Underspeed	Cruise trim threshold	Bypassed in glide
LF correction	Always	Disabled in glide
Integrator transitions	No reset	$\times 0.5$ on entry and exit

Part III

Fixed-Wing Position Control

Chapter 3

FixedwingPositionControl: Soaring Dispatch and Guidance

3.1 Original Architecture

3.1.1 Module Role

`FixedwingPositionControl` runs at ~ 50 Hz and:

1. reads estimates and the navigator setpoint triplet;
2. calls NPFG to compute roll (or reads loiter centre from navigator);
3. calls TECS via `tecs_update_pitch_throttle`;
4. publishes the attitude setpoint.

3.1.2 NPFG Guidance

Track-error angle η from velocity to path tangent drives lateral acceleration:

$$a_{\text{lat}} = \frac{2V_a^2}{L} \sin \eta \quad (3.1)$$

Roll setpoint:

$$\phi^* = \arctan\left(\frac{a_{\text{lat}}}{g}\right) \quad (3.2)$$

3.1.3 `adapt_airspeed_setpoint`

Floor-clamped EAS setpoint accounting for load factor:

$$V_{\text{EAS}}^* = \text{clamp}(V_{\text{sp}}, V_{\text{stall}}\sqrt{n_z}, V_{\text{EAS,max}}) \quad (3.3)$$

3.2 Soaring Mode Dispatch Logic

3.2.1 Mode Derivation

Each cycle in `control_auto()`:

```

1  const bool glide_cmd    = (soaring_cmd == SOARING_GLIDE_FIXED
2                               || soaring_cmd == SOARING_GLIDE_POLAR);
3  const bool thermal_cmd  = (soaring_cmd == SOARING_THERMAL_LOITER
4                               || soaring_cmd == SOARING_THERMAL_BANK);
5  const bool soaring_allowed = (soaring_cmd != SOARING_OFF)
6                                && (_current_altitude > _param_fw_alt_min.get()
7                                    || _soaring_local_override);
8  const bool effective_glide = soaring_allowed
9                                && (glide_cmd || _alt_max_reached);
10 const bool effective_thermal = soaring_allowed && thermal_cmd
11                                && !_alt_max_reached;
12 const bool effective_thermal_bank = effective_thermal
13                                && (soaring_cmd == SOARING_THERMAL_BANK);

```

3.2.2 Altitude Ceiling and Floor

Floor check:

$$h < h_{\min} \Rightarrow \text{soaring_allowed} = \text{false} \quad (3.4)$$

Ceiling:

$$h \geq h_{\max} \Rightarrow \text{_alt_max_reached} = \text{true} \quad (3.5)$$

$$h < h_{\max} - h_{\text{hyst}} \Rightarrow \text{_alt_max_reached} = \text{false} \quad (3.6)$$

3.3 Throttle Ramps

3.3.1 Entry Ramp (Powered $\rightarrow$ Glide)

$$\delta_t(t) = \delta_{t,\text{pre}} \left(1 - \frac{t - t_s}{T_{\text{ramp}}} \right), \quad t \in [t_s, t_s + T_{\text{ramp}}] \quad (3.7)$$

3.3.2 Exit Ramp (Glide $\rightarrow$ Powered)

$$\delta_t(t) = \delta_{t,\text{TECS}} \frac{t - t_e}{T_{\text{ramp}}}, \quad t \in [t_e, t_e + T_{\text{ramp}}] \quad (3.8)$$

Both ramps use $T_{\text{ramp}} = \text{FW_GLIDE_RAMP_T}$.

3.4 Airspeed Setpoints per Mode

Mode	Airspeed	Computation
1	Fixed	$V^* = \text{FW_GLIDE_AIRSPD}$ (or airspeed_cmd)
2	Polar	$V^* = \sqrt{b/a}$, computed per cycle
3,4	Hold	$V^* = \text{FW_GLIDE_AIRSPD}$

3.5 Mode 3: Thermal Loiter via Navigator

3.5.1 Loiter Radius

Minimum safe radius at bank ϕ , speed V_{TAS} :

$$R_{\min} = \frac{V_{\text{TAS}}^2}{g \tan \phi} \quad (3.9)$$

Effective radius:

$$R_{\text{eff}} = \max(R_c, R_{\min}) \quad (3.10)$$

3.5.2 Signed Radius Encoding

$$\text{param3} = \begin{cases} +R_{\text{eff}} & \text{clockwise} \\ -R_{\text{eff}} & \text{counter-clockwise} \end{cases} \quad (3.11)$$

3.5.3 Pre-Roll Safety (T3)

Before first DO_REPOSITION:

$$V_{\text{EAS,current}} \geq 1.05 V_{\text{stall}} \sqrt{n_z} \quad (3.12)$$

If not satisfied, the command is delayed until speed is sufficient.

3.6 Mode 4: Direct Bank-Angle Thermal Guidance

3.6.1 Why Bypass NPFG

Mode 4 bypasses the `switch(position_sp_type)` block entirely. Calling NPFG first would contaminate TECS with loiter-geometry-adjusted airspeed. By calling TECS directly and composing the quaternion manually, both pitch and roll are under explicit control.

3.6.2 Roll Sign Convention

$$\phi^* = s |\phi_{\text{cmd}}|, \quad s = \begin{cases} +1 & \text{loiter_clockwise} = \text{true} \\ -1 & \text{loiter_clockwise} = \text{false} \end{cases} \quad (3.13)$$

3.6.3 Attitude Setpoint Composition

$$\mathbf{q}^* = \text{Euler}(\phi^*, \theta_{\text{TECS}}, \psi_{\text{current}}) \quad (3.14)$$

```
1 const float bank_rad = bank_sign * math::radians(fabsf(  
    _soaring_bank_angle_cmd_deg));  
2 const Quatf q_bank(Eulerf(bank_rad, get_tecs_pitch(), _yaw));  
3 q_bank.copyTo(_att_sp.q_d);  
4 _att_sp.thrust_body[0] = get_tecs_thrust();
```

3.6.4 Summary

Feature	Original	Modified
Mode encoding	Two booleans	Single integer
Entry ramp	Instant cut	Smooth (T_{ramp})
Exit ramp	Instant	Smooth (T_{ramp})
Mode 4 roll	Not present	Direct bank-angle
Turn direction	Not supported	Signed radius + bool
Pre-roll safety	Not present	Banked stall check

Part IV

Commander: CLI Mode Switching

Chapter 4

Commander Soaring Commands

4.1 Original System

In the original Music codebase, Commander had no soaring-specific CLI commands. Soaring behaviour required manual parameter changes via a GCS.

4.2 Modified System: CLI Commands

4.2.1 Commands Added

```
1 commander soar:off      # mode 0 -- powered flight
2 commander soar:glide    # mode 1 -- fixed-EAS glide
3 commander soar:polar    # mode 2 -- polar best-glide
4 commander soar:thermal  # mode 3 -- navigator loiter
5 commander soar:bank     # mode 4 -- direct bank angle
```

4.2.2 Command Structure

Each command publishes `VEHICLE_CMD_CUSTOM_0` with `param1 = (float)soaring_mode`.

4.2.3 Local-Override Flag

CLI commands set `_soaring_local_override = true`, which:

- bypasses altitude floor check;
- disables staleness watchdog;
- is cleared on disarm or explicit `soar:off`.

Part V

Navigator: Loiter Direction Control

Chapter 5

Navigator DO_REPOSITION Extension

5.1 Original Handler

```
1 if (PX4_ISFINITE(cmd.param3) && cmd.param3 > 0) {  
2     rep->current.loiter_radius = cmd.param3;  
3     // direction never set from param3  
4 }
```

Listing 5.1: Original – positive radius only

5.2 Modified Handler

```
1 if (PX4_ISFINITE(cmd.param3) && fabsf(cmd.param3) > FLT_EPSILON) {  
2     rep->current.loiter_radius = fabsf(cmd.param3);  
3     rep->current.loiter_direction_counter_clockwise = (cmd.param3 < 0.f)  
4     ;  
5 }
```

Listing 5.2: Modified – signed radius encodes direction

5.2.1 Circular Flight Mathematics

Angular rate:

$$\omega = \frac{V_{TAS}}{R} \quad (5.1)$$

Turn direction:

$$\omega = \begin{cases} +|V/R| & \text{CCW (NED)} \\ -|V/R| & \text{CW (NED)} \end{cases} \quad (5.2)$$

Encoding:

$$\text{loiter_direction_counter_clockwise} = (\text{param3} < 0) \quad (5.3)$$

5.3 Direction-Change Trigger

A new `direction_changed` flag forces resend when only `loiter_clockwise` changes:

```
1 const bool direction_changed =  
2     (_autosoaring_control.loiter_clockwise != _soaring_last_clockwise);  
3 const bool pos_changed = (first_activation && ok) ||  
4     explicit_pos_changed  
    || direction_changed;
```

After sending, `_soaring_last_clockwise` is latched to prevent repeated re-sends.

Part VI

ROMFS and Parameter Definitions

Chapter 6

Soaring Parameters and Airframe Configuration

6.1 Parameter System Overview

PX4 parameters are compiled into flash with defaults via `PARAM_DEFINE_*` macros. The ROMFS airframe script overrides defaults for a specific vehicle.

6.2 Removed Parameter: `FW_GLIDE_MODE`

The parameter `FW_GLIDE_MODE` (integer, 0=fixed, 1=polar) was removed. Its function is now encoded in `soaring_mode = 1` or `2`.

6.3 New Parameters

6.3.1 `FW_GLIDE_AIRSPD` (10.0 m/s EAS)

Fixed glide airspeed and thermalling reference speed.

$$V_{\text{TAS}}^* = \sqrt{\frac{\rho_0}{\rho}} \times 10.0 \text{ m/s} \quad (6.1)$$

6.3.2 `FW_POLAR_A` (0.003 s/m) and `FW_POLAR_B` (0.5 m/s)

Parabolic polar: $w_s = aV^2 + b$.

Best-glide speed:

$$V_{\text{polar}}^* = \sqrt{0.5/0.003} \approx 12.9 \text{ m/s} \quad (6.2)$$

Maximum glide ratio:

$$(L/D)_{\text{max}} = 1/(2\sqrt{0.003 \times 0.5}) \approx 12.9 \quad (6.3)$$

6.3.3 FW_GLIDE_I_DECAY (10.0 s)

Throttle integrator decay constant:

$$I_t(t) = I_t(0) e^{-t/10} \Rightarrow I_t(30 \text{ s}) = I_t(0) e^{-3} \approx 0.05 I_t(0) \quad (6.4)$$

6.3.4 FW_GLIDE_RAMP_T (3.0 s)

Throttle ramp duration. At 50 Hz: 150 cycles. Rate: $\delta_t(0)/T = 0.133/\text{s}$.

6.3.5 FW_THERMAL_BANK (40 deg)

Default bank angle. Load factor and minimum radius:

$$n_z = 1/\cos 40^\circ = 1.305 \quad (6.5)$$

$$R_{\min} = \frac{V^2}{g \tan 40^\circ} = \frac{10^2}{9.81 \times 0.839} \approx 12.2 \text{ m} \quad (6.6)$$

6.3.6 FW_ALT_MIN (50 m), FW_ALT_MAX (200 m), FW_ALT_HYST (20 m)

Soaring altitude envelope with ceiling hysteresis.

6.4 ROMFS Airframe Defaults

Added to ROMFS/px4fmu_common/init.d/airframes/4008_gz_advanced_plane:

```
1 param set FW_GLIDE_AIRSPD 10.0
2 param set FW_POLAR_A 0.003
3 param set FW_POLAR_B 0.5
4 param set FW_GLIDE_I_DECAY 10.0
5 param set FW_GLIDE_RAMP_T 3.0
6 param set FW_THERMAL_BANK 40.0
7 param set FW_ALT_MIN 50.0
8 param set FW_ALT_MAX 200.0
9 param set FW_ALT_HYST 20.0
```

6.5 Summary

Aspect	Original	Modified
Soaring parameters	None	9 new parameters
FW_GLIDE_MODE	Defined	Removed
Airframe defaults	None	Added to 4008 airframe

Appendix A

ROS 2 Command Reference

```
1 # Mode 0: OFF (powered)
2 ros2 topic pub --once /fmu/in/autosoaring_control \
3   px4_msgs/msg/AutosoaringControl \
4   "{timestamp:␣0,␣soaring_mode:␣0,␣loiter_radius_m:␣0.0,␣loiter_lat:␣
5     0.0,
6     ␣␣␣␣loiter_lon:␣0.0,␣airspeed_cmd:␣0.0,␣bank_angle_cmd:␣0.0,
7     ␣␣␣␣loiter_clockwise:␣true,␣soaring_phase:␣0}"
8
9 # Mode 1: Fixed-EAS Glide at 10 m/s
10 ros2 topic pub --once /fmu/in/autosoaring_control \
11   px4_msgs/msg/AutosoaringControl \
12   "{timestamp:␣0,␣soaring_mode:␣1,␣loiter_radius_m:␣0.0,␣loiter_lat:␣
13     0.0,
14     ␣␣␣␣loiter_lon:␣0.0,␣airspeed_cmd:␣10.0,␣bank_angle_cmd:␣0.0,
15     ␣␣␣␣loiter_clockwise:␣true,␣soaring_phase:␣1}"
16
17 # Mode 2: Polar best-glide
18 ros2 topic pub --once /fmu/in/autosoaring_control \
19   px4_msgs/msg/AutosoaringControl \
20   "{timestamp:␣0,␣soaring_mode:␣2,␣loiter_radius_m:␣0.0,␣loiter_lat:␣
21     0.0,
22     ␣␣␣␣loiter_lon:␣0.0,␣airspeed_cmd:␣0.0,␣bank_angle_cmd:␣0.0,
23     ␣␣␣␣loiter_clockwise:␣true,␣soaring_phase:␣1}"
24
25 # Mode 3: Thermal loiter, CW, 40-deg bank, R=15m
26 ros2 topic pub --once /fmu/in/autosoaring_control \
27   px4_msgs/msg/AutosoaringControl \
28   "{timestamp:␣0,␣soaring_mode:␣3,␣loiter_radius_m:␣15.0,␣loiter_lat:␣
29     0.0,
30     ␣␣␣␣loiter_lon:␣0.0,␣airspeed_cmd:␣0.0,␣bank_angle_cmd:␣40.0,
31     ␣␣␣␣loiter_clockwise:␣true,␣soaring_phase:␣2}"
32
33 # Mode 3: Thermal loiter, CCW
34 ros2 topic pub --once /fmu/in/autosoaring_control \
35   px4_msgs/msg/AutosoaringControl \
36   "{timestamp:␣0,␣soaring_mode:␣3,␣loiter_radius_m:␣15.0,␣loiter_lat:␣
37     0.0,
38     ␣␣␣␣loiter_lon:␣0.0,␣airspeed_cmd:␣0.0,␣bank_angle_cmd:␣40.0,
39     ␣␣␣␣loiter_clockwise:␣false,␣soaring_phase:␣2}"
40
41 # Mode 4: Direct bank angle, 40-deg, CW
```

```

37 ros2 topic pub --once /fmu/in/autosoaring_control \
38   px4_msgs/msg/AutosoaringControl \
39   "{timestamp: 0, soaring_mode: 4, loiter_radius_m: 0.0, loiter_lat: 0.0,
40    loiter_lon: 0.0, airspeed_cmd: 0.0, bank_angle_cmd: 40.0,
41    loiter_clockwise: true, soaring_phase: 3}"

```

Listing A.1: ROS 2 test commands for each soaring mode